

Paralelización con MPI para el método de búsqueda aleatorio para búsqueda de mínimos globales con tunelamiento

Brayhan Pérez Rendon*
Universidad de Antioquia

(FACOM)

(Dated: May 2, 2025)

Este trabajo desarrolla un método estocástico de optimización global que combina muestreo MonteCarlo de N partículas, selección de las M mejores, y un mecanismo de tunelamiento para escapar de óptimos locales, todo implementado en Python y paralelizado con MPI sobre múltiples núcleos; las pruebas en dos equipos (24 y 8 núcleos) muestran que la versión paralela reduce el tiempo de cómputo hasta ocho veces frente a la secuencial y escala linealmente al aumentar el número de partículas, lo que valida la eficacia y robustez del enfoque, y abre caminos para comparar otras librerías de paralelización, optimizar la comunicación entre procesos y estudiar el impacto de los parámetros de exploración en la convergencia.

I. INTRODUCCIÓN

La optimización de funciones complejas mediante métodos estocásticos se ha consolidado como una estrategia versátil para hallar mínimos globales en problemas de alta dimensionalidad. Partiendo de trabajos previos—incluyendo la aplicación de Particle Swarm Optimization (PSO) y del algoritmo School Fish Search—, en este proyecto se propone un método basado en muestreo Monte Carlo con un mecanismo de tunelamiento para superar el estancamiento en mínimos locales [?].

El algoritmo arranca generando N partículas uniformemente distribuidas en un dominio N -dimensional. A partir de estas, se seleccionan las M partículas con los valores más bajos de la función objetivo, cumpliendo

$$f(x_m) < f(x), \quad m = 1, \dots, M,$$

y se redefine la región de muestreo alrededor de estas partículas destacadas. Para evitar la trampa en óptimos locales, se incorpora un operador de tunelamiento inspirado en la formulación de Levy y Montalvo (1985), que reconstruye una función auxiliar

$$T(x) = \frac{f(x) - f(x^*)}{\|x - x^*\|^{2\eta}},$$

donde x^* es el candidato dominante previo y η regula la amplitud de exploración [?].

Esta aproximación ofrece cuatro ventajas principales:

1. **Robustez** frente a múltiples mínimos locales, gracias al tunelamiento.
2. **Eficiencia** en el número de evaluaciones de la función objetivo, al centrar el muestreo en regiones prometedoras.
3. **Paralelización natural**, explotando la independencia en la generación de partículas.

4. **Adaptabilidad**, ya que parámetros como “Size ite” (factor de reducción de región) y η se pueden calibrar para distintas familias de funciones benchmark.

II. MÉTODOS

Para implementar el algoritmo en Python, se emplea MPI (Message Passing Interface) junto a una plataforma Intel Core i9 vPro con 24 núcleos y 64 GB de RAM. El dominio de búsqueda y las partículas seleccionadas se distribuyen equitativamente entre todos los procesos mediante `MPI_Scatter`. Cada núcleo:

1. Recibe un subconjunto de partículas.
2. Evalúa la función objetivo localmente y determina la mejor partícula de su región.
3. Envía el resultado a la raíz, que consolida la información, ajusta el tamaño del dominio y vuelve a repartir las nuevas partículas.

Este esquema se repite hasta convergencia o hasta agotarse el presupuesto de iteraciones. Para comparar escalabilidad, se realizaron pruebas en un segundo equipo con Intel Core i5 de 12^a generación y 8 núcleos, variando N en {10, 20, 50, 100} partículas muestreadas.

III. IMPLEMENTACIÓN Y EJECUCIÓN

El código principal se ejecuta con:

```
mpiexec -n 8 python OptimizadorconMPIcorregido.py
```

En el segundo equipo se mantuvo la misma configuración para evaluar la influencia del número de núcleos y las partículas seleccionadas (M) sobre el tiempo de cómputo.

* Also at Instituto de Física, Maestría en Física

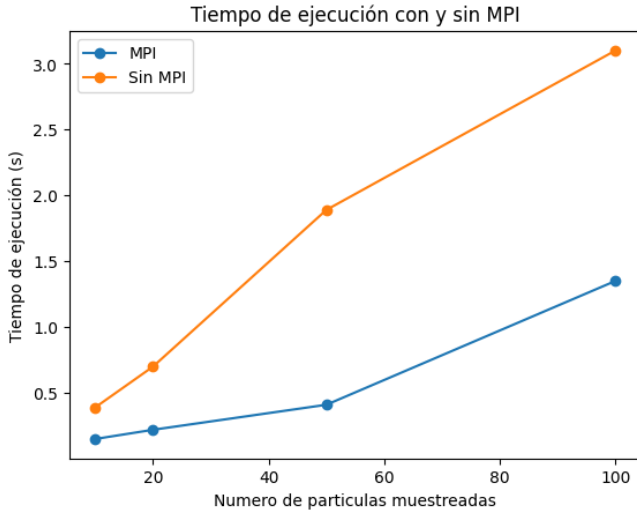


FIG. 1: Caption

IV. RESULTADO

- **Sin paralelizar:** con $N = 100$ y $M = 24$ en el primer equipo, el tiempo medio fue de 91 s.
- **Paralelizado:** bajo las mismas condiciones, el tiempo cayó a 14 s, lo que representa una reducción cercana a $1/8$.
- En el segundo equipo, seleccionar solo $M = 8$

partículas disminuyó aún más el tiempo, pues el cómputo de cada ciclo escala con $M \times N$.

- Al analizar promedios sobre 50 ejecuciones, se observó un crecimiento lineal del tiempo conforme aumenta N , y una clara ventaja de la versión paralela al incrementarse la complejidad de la función.

V. CONCLUSIONES

La combinación de muestreo Monte Carlo con tunelamiento demuestra alta eficacia para evitar mínimos locales y converger rápidamente hacia el mínimo global. MPI resultó ser una herramienta poderosa para paralelizar el proceso de evaluación de partículas, aunque su integración requiere una invasión profunda del flujo de control del algoritmo. En futuras etapas será conveniente:

- Comparar con otras librerías de paralelización (p.ej., `multiprocessing` o Condor).
- Ampliar los estudios de convergencia, explorando distintos valores de “Size ite” y η en más funciones de prueba.
- Optimizar la gestión de comunicación entre procesos para reducir la latencia en la raíz.

-
- [1] R. Y. Rubinstein and D. P. Kroese, *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning* (Springer, 2004).
 - [2] Ansys, <https://www.ansys.com/> (2025), accedido: 25 de marzo de 2025.
 - [3] J. Kennedy and R. Eberhart, Particle swarm optimization, in *Proceedings of ICNN'95 - International Conference on Neural Networks*, Vol. 4 (1995) pp. 1942–1948 vol.4.
 - [4] C. Bastos-Filho, F. Lima Neto, A. Lins, A. Nascimento, and M. Lima, A novel search algorithm based on fish school behavior (2008) pp. 2646 – 2651.
 - [5] A. V. Levy and A. Montalvo, The tunneling algorithm for the global minimization of functions, *SIAM Journal on Scientific and Statistical Computing* **6**, 15 (1985), <https://doi.org/10.1137/0906002>.
 - [6] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard, Version 4.1*, MPI Forum (2023), disponible en línea.